



FineWeb: decanting the web for the finest text data at scale



The  FineWeb dataset, clustered and annotated with educational score labels

AUTHORS

Guilherme Penedo, Hynek Kydičėk, Loubna Ben Allal, Anton Lozhkov,
Colin Raffel, Leandro Werra, Thomas Wolf

AFFILIATION

HuggingFace

PUBLISHED

May 31, 2024

Table of contents

Web data

- Finding the raw data
- Processing at scale
- What is good data?
- Ablations and evaluation setup

The FineWeb recipe

- Starting point: text extraction
- Base filtering
- Deduplicating the data
 - Why deduplicate?
 - Our deduplication parameters
 - More deduplication is always better, right?
 - Taking a step back: individual dump dedup
 - A note on measuring the effect of deduplication
 - Other (failed) global approaches
- Additional quality filtering
 - C4: A dataset that has stood the test of time
 - A statistical approach to develop heuristic filters
- The final  FineWeb dataset
 - Comparisons with other web-scale datasets

FineWeb-Edu

- Annotating for educational quality at scale
- Training a classifier
- Filtering and results

Bonus: CommonCrawl over time

- Benchmark performance by crawl
- Synthetic data

Conclusion and looking forward

The performance of a large language model (LLM) depends heavily on the quality and size of its pretraining dataset. However, the pretraining datasets for state-of-the-art open LLMs like Llama 3^[1] and Mixtral^[2] are not publicly available and very little is known about how they were created.

Recently, we released  **FineWeb**, a new, large-scale (**15-trillion tokens, 44TB disk space**) dataset for LLM pretraining. FineWeb is derived from 96 CommonCrawl snapshots and produces **better-performing LLMs than other open pretraining datasets**. To bring more clarity in machine learning and advance the open understanding of how to train good quality large language models, we carefully documented and ablated all of the design choices used in FineWeb, including in-depth investigations of deduplication and filtering strategies. The present long form report is a deep dive in how to create a large and high-quality web-scale dataset for LLM pretraining. The dataset itself,  FineWeb, is available [here](#).

In this report we also introduce  **FineWeb-Edu**, a subset of FineWeb constructed using scalable automated high-quality annotations for educational value, and which outperforms all openly accessible web-datasets on a number of educational benchmarks such as MMLU, ARC, and OpenBookQA.  FineWeb-Edu is available in two sizes/filtering-level: **1.3 trillion (very high educational content)** and **5.4 trillion (high educational content)** tokens (all tokens are measured with GPT2 tokenizer ^[3]). You can download it [here](#).

Both datasets are released under the permissive [ODC-By 1.0 license](#)

TLDR: This blog covers a discussion on processing and evaluating data quality at scale, the  FineWeb recipe (listing and explaining all of our design choices), and the process followed to create its  FineWeb-Edu subset.

Reading time: 45 min. For the best reading experience, we recommend not using a mobile phone.

We are extremely thankful to the whole [distill.pub](#) team (Christopher Olah, Shan Carter, Ludwig Schubert in particular) for creating the template on which we based this blog post. Thanks also for inspiring us with exquisitely crafted articles and blog posts.

Web data

Finding the raw data

A common question often asked regarding web datasets used to train LLMs is “where do they even get all that data?”. There are generally two options:

- you either crawl it yourself, like companies such as OpenAI or Anthropic (among others) do (see [here](#) and [here](#))
- you use a public repository of crawled webpages, like the one maintained by the non-profit

To build 🦋 FineWeb, following what has been done in the past by a number of LLM training teams, we used [CommonCrawl](#) (CC) as a starting point. The Common Crawl non-profit organization has been crawling the web since 2007 and releases a new crawl containing 200 to 400 TiB of textual content obtained via automatic web crawling usually every 1 or 2 months.

As an example, the latest CC crawl (April 2024) contains 2.7 billion web pages, totaling 386 TiB of uncompressed HTML text content ¹. Ninety-six crawls have been released since 2013 and 3 crawls from 2008 to 2012, which are in a different (older) format. ²

Processing at scale

Given the sheer size of the data involved, one of the main challenges we had to overcome was having a modular, scalable codebase that would allow us to quickly iterate on our processing decisions and easily try out new ideas, while appropriately parallelizing our workloads and providing clear insights into the data.

For this purpose, we developed [datatrove](#)^[4], an open-source data processing library that allowed us to seamlessly scale our filtering and deduplication setup to thousands of CPU cores. All the data processing steps involved in the creation of 🦋 FineWeb used this [library](#). You will find the exact scripts we used in the [datatrove repository](#).

What is good data?

This is probably the main question to keep in mind when creating a dataset. In most contexts and, in particular, in the context of large language model pretraining ³, "high quality" is not a very well defined term^[5] ^[6], and not even a property of documents that can always be clearly perceived through direct human observation alone.^[7]

It is still common to train a model on a given corpus considered "clean" (typically wikipedia ⁴) and use it to check the perplexity on the dataset that we were trying to curate^[8]. Unfortunately this does not always correlate with improved performance on a set of downstream tasks of interest^[9], and as a result another often used approach is to train small models ⁵ on a representative subset of our dataset and evaluate them on a set of evaluation tasks. Small models are used because training costs and time are a function of model size. In this second approach, it is important to choose a diverse and representative set of dataset-evaluation tasks and try not to overfit to any one individual benchmark as it would risk hurting the generality of the obtained LLM after pretraining.

Yet another way to compare different datasets would be to train a model on each dataset and have humans rate and compare the generations of the models (like on the [LMSYS Chatbot Arena](#))^[10]. This would arguably provide the most reliable results in terms of representing real model usage, but getting ablation results this way is unfortunately expensive and slow. It also often requires for the models to have undergone an instruction finetuning stage to acquire conversational capabilities, as pretrained models are not directly designed to follow instructions and are thus much more sensitive to prompt details.^[11]

In this work, we went with the approach of training small models and evaluating them on a set of "early-signal" benchmark tasks. We believe this is a reasonable proxy for the quality of the data used to train these models, when keeping in mind the above-mentioned caveat around overfitting on the evaluation benchmarks.

Ablations and evaluation setup

To compare the impact of a given processing step, we trained two models on two versions of the dataset, one version processed with the extra step (the one we wish to evaluate) and another version with this step ablated (cut/removed). Apart from the data, these two models would be otherwise identical: the same number of parameters, architecture hyper-parameters, and trained on an equal number of randomly sampled tokens from each version of the data, for a single epoch — the only difference being thus the training data. We then evaluated each model on the same set of tasks and compared average scores.

Our ablation models were trained using [nanotron](#). Our "ablation models" have 1.82B parameters (including embeddings), used the Llama architecture with a 2048 sequence length, a global batch size of ~2 million tokens, and the GPT2 tokenizer. For most ablations we trained on ~28B tokens (roughly the Chinchilla^[12] optimal training size for this model size). To confirm relative performance improvements after each step of filtering we conducted longer training runs on 350 billion tokens as mentioned further below.

We evaluated the models using [lighteval](#). We carefully selected a set of benchmark for ablations by selecting benchmarks that would provide good signal at a relatively small scale

We'll make the configuration to reproduce these ablation models available soon in Nanotron.

("small" models trained on only "a few billion" tokens). We generally used the following criteria to select these benchmarks among all the benchmarks available in `lighteval`:

- small variance between runs trained on different samplings of the same dataset: we want our runs on a subset of the data to be representative of the whole dataset, and the resulting scores to be, in the limit of what is possible, less sensitive to exact data point choices than to our filter's effect
- performance increasing monotonically (or close) over a training run: ideally, as the number of seen tokens increases, the performance on a high-signal benchmark should not decrease (which would be indicative of unreliable results at a small scale)
- performance above random baseline for this task by at least a few standard deviations: given our small ablation models and trainings we usually don't reach extremely high scores on any benchmark, but we want to make sure that the scores we get are above random noise.

After consideration, we selected the following list of benchmarks:

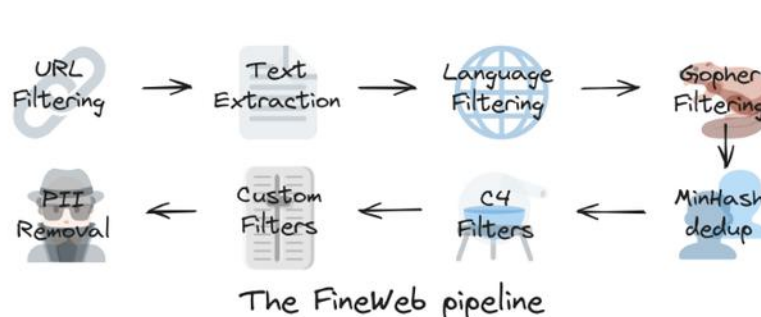
- CommonSense QA^[13]
- HellaSwag^[14]
- OpenBook QA^[15]
- PIQA^[16]
- SIQA^[17]
- WinoGrande^[18]
- ARC^[19]
- MMLU^[20]

To ensure our checkpoint evaluation stayed within a limited timeframe, we capped the longer benchmarks at 1000 samples (wall-clock evaluation taking less than 5 min on a single node of 8 GPUs - done in parallel to the training).

You can find the full list of tasks and prompts we used [here](#).

The 🍷 FineWeb recipe

In the next subsections we will explain each of the steps taken to produce the FineWeb dataset.



You can find a fully reproducible datatrove config [here](#).

Starting point: text extraction

CommonCrawl data is available in two main formats: WARC and WET. **WARC** (Web ARchive format) files contain the raw data from the crawl, including the full page HTML and request metadata. **WET** (WARC Encapsulated Text) files provide a text only version of those websites.

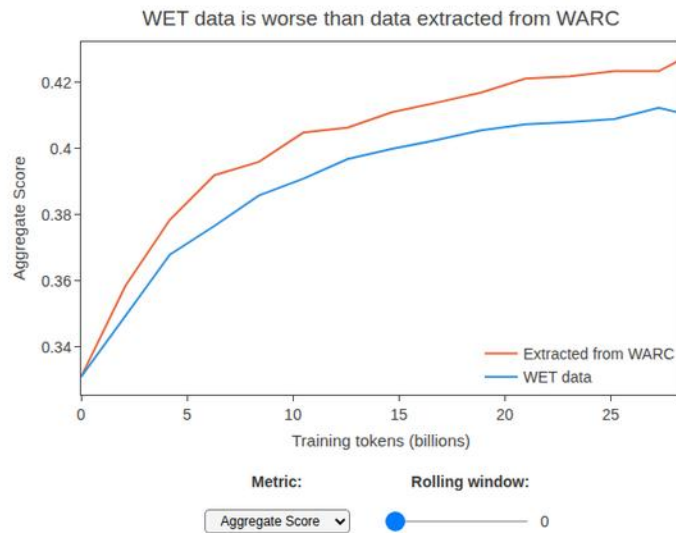
A large number of datasets take the WET files as their starting point. In our experience the default text extraction used by Common Crawl to create these WET files is suboptimal for the goals of LLM pretraining⁶ and there are a variety of open-source libraries that provide better text extraction. We extracted the text content from the WARC files using the `trafilatura` library^[21], which from visual inspection of the results provided good quality extraction when compared to other libraries.

You can find a benchmark comparing several text extraction libraries [here](#).

To validate this decision, we processed the 2019-18 dump directly using the WET files and with text extracted from WARC files using `trafilatura`⁷. We applied the same processing to each one (our base filtering+minhash, detailed below) and trained two models. While the resulting dataset is about 25% larger for the WET data (around 254 billion tokens), it proves to be of much worse quality than the one that used `trafilatura` to extract text from WARC files.

le or much worse quality than the one that used traindata to extract text from WARC files (which is around 200 billion tokens). Visual inspection of some samples confirms that many of these additional tokens on the WET files are unnecessary page boilerplate.

It is important to note, however, that text extraction is one of the most costly steps of our processing, so we believe that using the readily available WET data could be a reasonable trade-off for lower budget teams.



Base filtering

Filtering is an important part of the curation process. It consists in removing part of the data (be it words, lines, or even full documents) that lowers the performance of the model and is thus deemed to be “lower quality” in our eval-driven process of dataset crafting.

As a basis for our filtering we used part of the setup from RefinedWeb^[22]. Namely, we:

- Applied URL filtering using a [blocklist](#) to remove adult content
- Applied a [fastText language classifier](#)^[23] ^[24] to keep only English text with a score ≥ 0.65
- Applied quality and repetition filters from MassiveText^[25] (using the default thresholds)

After applying this filtering to each of the text extracted dumps (there are currently 96 dumps) we obtained roughly 36 trillion tokens of data⁸.

Deduplicating the data

Deduplication is one of the most important steps when creating large web datasets for LLM pretraining. Methods to deduplicate datasets attempt to identify and remove redundant/repeated data from the dataset.

WHY DEDUPLICATE?

The web has many aggregators, mirrors, templated pages or just otherwise repeated content spread over different domains and webpages. Sometimes, these duplicated pages can even be introduced by the crawler itself, when different links point to the same page.

Removing these duplicates (deduplicating) has been correlated with improvements in model performance^[26] and a reduction in memorization of pretraining data^[27], which might allow for better generalization. Additionally, the performance uplift obtained through deduplication can be equated to increased training efficiency: by removing duplicated content, a model can reach the same performance level with fewer training iterations – or equivalently, for a given number of training tokens, a model will have seen more diverse data.^[28] ^[29]

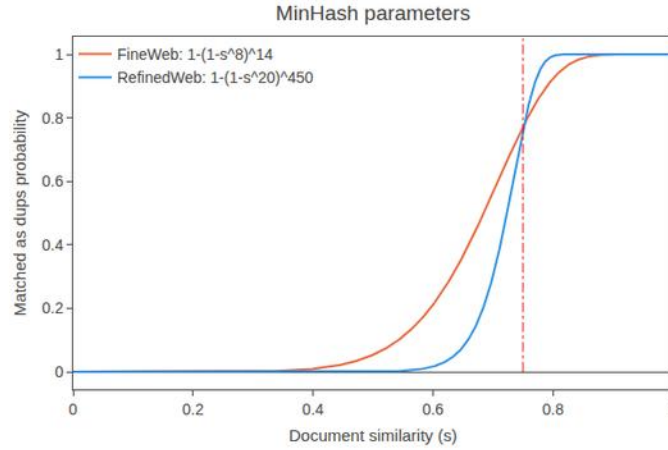
There are different ways to identify and even define duplicated data. Common approaches rely on hashing techniques to speed up the process, or on building efficient data structures to index the data (like suffix arrays). Methods can also be “fuzzy”, by using some similarity metric to mark documents as duplicates, or “exact” by checking for exact matches between two documents (or lines, paragraphs, or whatever other granularity level being used)⁹.

OUR DEDUPLICATION PARAMETERS

Following RefinedWeb^[22], we decided to apply MinHash, a fuzzy hash based deduplication technique that scales efficiently to many CPU-nodes and allows us to tune similarity thresholds (by controlling the number and size of buckets) as well as the length of the subsequences considered (by controlling the n-gram size). We chose to collect each

subsequences considered (by controlling the n-gram size). We chose to collect each document's 5-grams¹⁰ and compute minhashes using 112 hash functions in total, split into 14 buckets of 8 hashes each — targeting documents that are at least 75% similar. Documents with the same 8 minhashes in any bucket are considered a duplicate of each other.

This would mean that for two documents with a similarity (s) of 0.7, 0.75, 0.8 and 0.85, the probability that they would be identified as duplicates would be 56%, 77%, 92% and 98.8% respectively ($1 - (1 - s^8)^{14}$). See the plot below for a match probability comparison between our setup with 112 hashes and the one from RefinedWeb, with 9000 hashes, divided into 450 buckets of 20 hashes (that requires a substantially larger amount of compute resources, as each individual hash must be computed, stored and then compared with hashes from other documents):



While the high number of hash functions in RefinedWeb allows for a steeper, more well defined cut off (documents with real similarity near the threshold are more likely to be correctly identified), we believe the compute and storage savings are a reasonable trade off.

It should also be noted that intra-document deduplication is already handled by our repetition filter, which removes documents with many repeated lines and paragraphs.

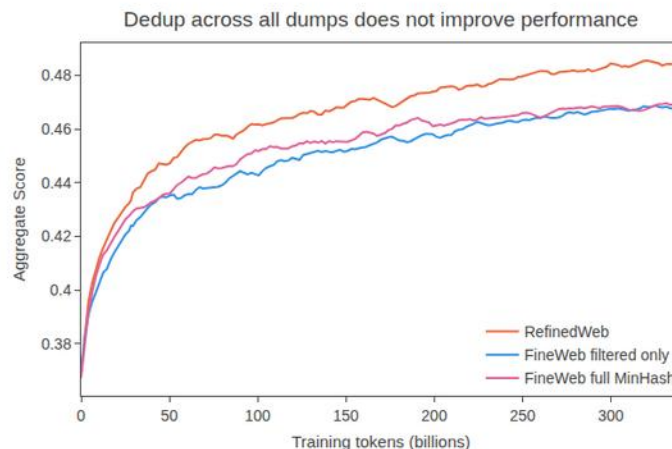
MORE DEDUPLICATION IS ALWAYS BETTER, RIGHT?

Initially, we were operating under the assumption that *more deduplication is always better*, so our first approach was to take the entire dataset (all 90+ dumps) and deduplicate them together as one big dataset using MinHash.

We did this in an iterative manner: starting with the most recent dump (which at the time was 2023-50) and proceeding chronologically until we reached the oldest crawl. We deduplicated each dump not only within itself, but removing any document matching any other documents in the previously processed dumps.

For instance, for the second most recent dump (2023-40 at the time), we deduplicated it against the most recent one in addition to within itself. As a result, the older the dumps, the larger the number of dumps it was deduplicated against and the more data we removed from it (indeed, in the oldest dumps, the deduplication step removed more than 90% of the base filtered data).

Deduplicating the dataset in this manner resulted in 4 trillion tokens of data, but, quite surprisingly to us, when training on a randomly sampled 350 billion tokens subset, our ablation models showed next to no improvement over a model trained on the non deduplicated data, scoring far below its predecessor RefinedWeb on our aggregate of tasks (see graph below).



Metric:

Rolling window:

Aggregate Score

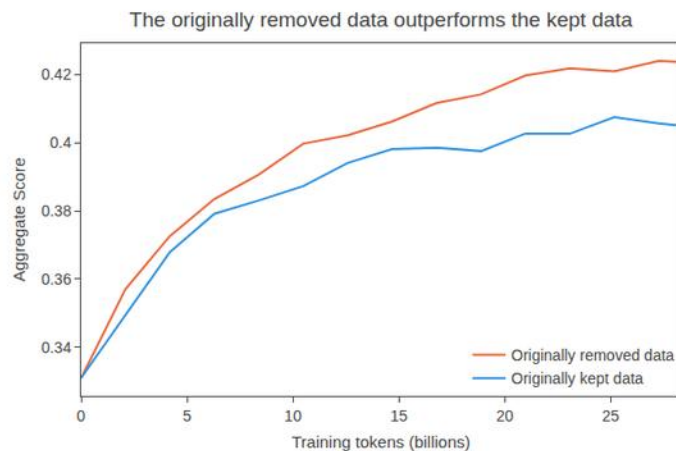
5

This challenged our assumption that more deduplication would inevitably result in higher benchmark scores, so we decided to take a closer look at one of the oldest dumps, dump 2013-48:

- pre deduplication, this dump had ~490 billion tokens
- after our iterative MinHash, ~31 billion tokens remained (94% of data had been removed)

As an experiment, we tried training two models on 28 billion tokens sampled from the following data from 2013-48:

- the fully deduplicated remaining ~31 billion tokens (*originally kept data*)
- 171 billion tokens obtained by individually deduplicating (without considering the other dumps) the ~460 billion tokens that had been removed from this dump in the iterative dedup process (*originally removed data*)¹¹



Metric:

Rolling window:

Aggregate Score

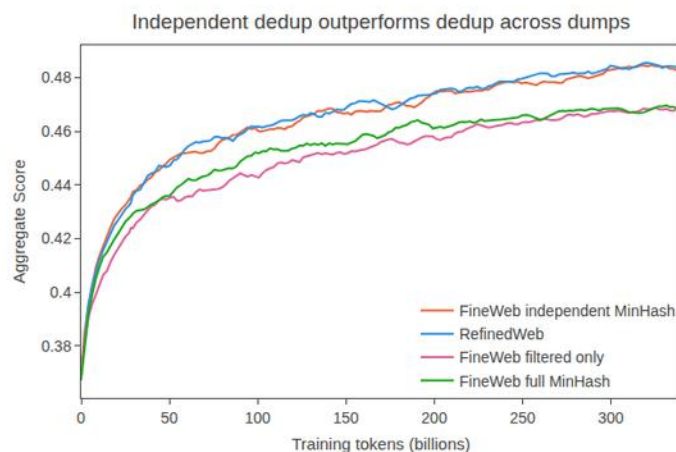
0

These results show that, for this older dump taken in isolation, the data that was kept (10% of the original data) was actually *worse* than the 90% of data we removed¹². This is also confirmed by visual inspection: *originally kept data* contains far more ads, lists of keywords and generally badly formatted text than *originally removed data*.

TAKING A STEP BACK: INDIVIDUAL DUMP DEDUP

We decided to experiment with an alternative approach: we deduplicated each dump with MinHash individually (independently of the other dumps). This resulted in 20 trillion tokens of data.

When training on a random sample from this dataset we see that it now matches RefinedWeb's performance (see curves below):



Metric:

Rolling window:

Aggregate Score

5

We hypothesize that the main improvement gained from deduplication is the removal of very large clusters that are present in every single dump (you will find some examples of these clusters in the RefinedWeb paper, each containing *hundreds of thousands* of documents) and

that further deduplication for clusters with a low number of duplicates (less than ~100 i.e. the number of dumps) actually harms performance: data that does not find a duplicate match in any other dump might actually be worse quality/more out of distribution (as evidenced by the results on the 2013-48 data).

While you might see some performance improvement when deduplicating a few dumps together, at the scale of the entire dataset (all the dumps), the effect from this upsampling of lower quality data side effect seems to be more impactful.

One possibility to consider is that as filtering quality improves, this effect may not be as prevalent, since the filtering might be able to remove some of this lower quality data. We also experimented with applying different, and often "lighter", deduplication approaches on top of the individually deduplicated dumps. You can read about them further below.

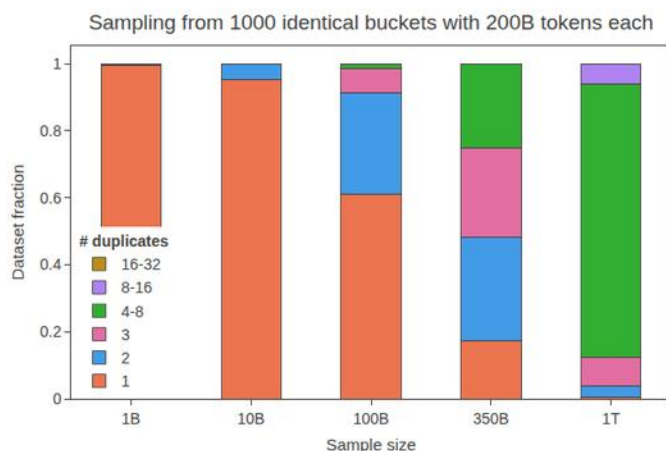
A NOTE ON MEASURING THE EFFECT OF DEDUPLICATION

Given the nature of deduplication, its effect is not always very visible in a smaller slice of the dataset (such as 28B tokens, the size we used for our filtering ablations). Furthermore, one must consider the fact that there are specific effects at play when deduplicating across all CommonCrawl dumps, as some URLs/pages are recrawled from one dump to the next.

To visualize the effect of scaling the number of training tokens on measuring deduplication impact, we considered the following (very extreme and unrealistic regarding the degree of duplication observed) theoretical scenario:

- there are 100 CommonCrawl dumps (roughly accurate)
- each dump has been perfectly individually deduplicated (every single document is unique in this dump)
- each dump is a perfect copy of each other (maximum possible duplication across dumps, effectively the worst case scenario)
- each dump has 200 billion tokens (for a total of 20 trillion, the resulting size of our individual dedup above)
- each dump is made up of documents of 1k tokens (200M documents per dump)

We then simulated uniformly sampling documents from this entire dataset of 20 trillion tokens, to obtain subsets of 1B, 10B, 100B, 350B and 1T tokens. In the image below you can see how often each document would be repeated.



For 1B almost all documents would be unique (#duplicates=1), despite the fact that in the entire dataset each document is repeated 100 times (once per dump). We start seeing some changes at the 100B scale (0.5% of the total dataset), with a large number of documents being repeated twice, and a few even 4-8 times. At the larger scale of 1T (5% of the total dataset), the majority of the documents are repeated up to 8 times, with some being repeated up to 16 times.

We ran our performance evaluations for the deduplicated data at the 350B scale, which would, under this theoretical scenario, be made up of a significant portion of documents duplicated up to 8 times. This simulation illustrates the inherent difficulties associated with measuring deduplication impact on the training of LLMs, once the biggest duplicate clusters have been removed.

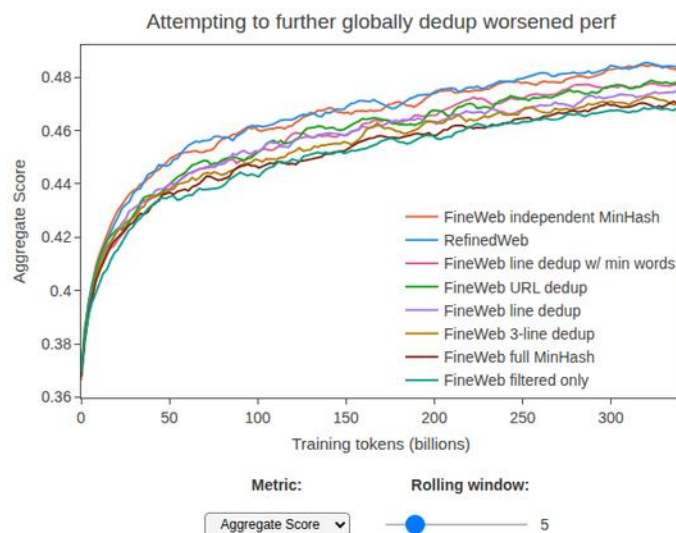
OTHER (FAILED) GLOBAL APPROACHES

To build on top of our newly found method (independently deduplicating each dump). We attempted to improve the performance by further deduplicating the independently minhash deduped 20 trillion tokens of data with alternative global (over all dumps) deduplication

methods. We explored the following approaches:

- URL deduplication, where we only kept one document per normalized (lowercased) URL (71.5% of tokens removed, 5.6T left) — *FineWeb URL dedup*
- Line deduplication:
 - remove all but 1 (randomly chosen) occurrence of each duplicated line (77.8% of tokens dropped, 4.4T left) — *FineWeb line dedup*
 - same as above, but only removing duplicate lines with at least 10 words and dropping documents with fewer than 3 sentences after deduplication (85% of tokens dropped, 2.9T left) — *FineWeb line dedup w/ min words*
 - remove all but 1 occurrence of each span of 3 duplicated lines with each number treated as 0 when finding duplicates, (80.9% of tokens removed, 3.7T left) — *FineWeb 3-line dedup*

The performance of the models trained on each of these was consistently worse (even if to different degrees) than that of the original independently deduplicated data:



Additional quality filtering

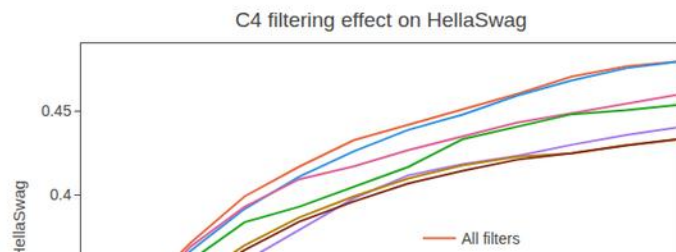
By this point we had reached the same performance of the previous work we attempted to reproduce and extend: RefinedWeb, using our base filtering and independent MinHash. Still, on our aggregate of tasks, another heavily filtered dataset, the C4 dataset^[30], still showed stronger performances on some benchmarks of our evaluation suite.

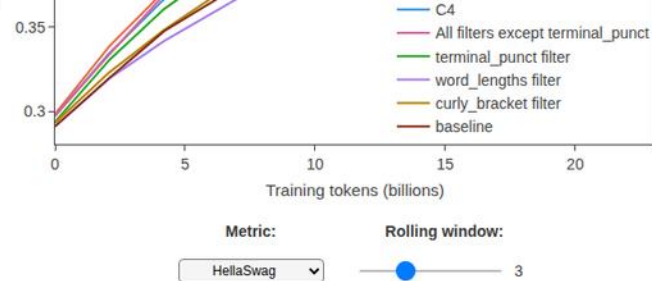
We therefore set out to find new filtering steps that would, at first, allow us to match the performance of C4 and, at a second stage, surpass it. A natural starting point was to look into the processing of C4 itself.

C4: A DATASET THAT HAS STOOD THE TEST OF TIME

The C4 dataset was first released in 2019. It was obtained from the 2019-18 CommonCrawl dump by removing non english data, applying some heuristic filters on both the line and document level, deduplicating on the line level, and removing documents containing words from a word blacklist.

Despite its age and limited size for current standards (around 175B gpt2 tokens), this dataset is, to this day, a common sub-set of typical LLM training, being used in models such as the relatively recent Llama1^[31]. This success is due to the strong performance that models trained on this dataset exhibit, excelling in particular on the Hellaswag benchmark^[14], one of the benchmarks in our “early signal” group with the highest signal-to-noise ratio. We experimented applying each of the different filters used in C4 to a baseline of the independently deduped FineWeb 2019-18 dump:





- applying "All filters" (drop lines not ending on punctuation marks, mentioning javascript and cookie notices + drop documents outside length thresholds, containing "lorem ipsum" or a curly bracket, { }) allows us to match C4's HellaSwag performance ("All filters" vs "C4" curves, respectively).
- The curly bracket filter, and the word lengths filter only give a small boost, removing 2.8% and 4.3% of tokens, respectively
- The terminal punctuation filter, by itself, gives the biggest individual boost, but removes *around 30%* of all tokens (!)
- The lorem_ipsum, javascript and policy rules each remove <0.5% of training tokens, so we did not train on them individually
- "All filters except the (very destructive) terminal_punct" performs better than terminal_punct by itself, while removing less in total (~7%)

We decided to apply all C4 filters mentioned above except the terminal punctuation one. We validated these results with a longer run, which you will find in a plot in the next section.

A STATISTICAL APPROACH TO DEVELOP HEURISTIC FILTERS

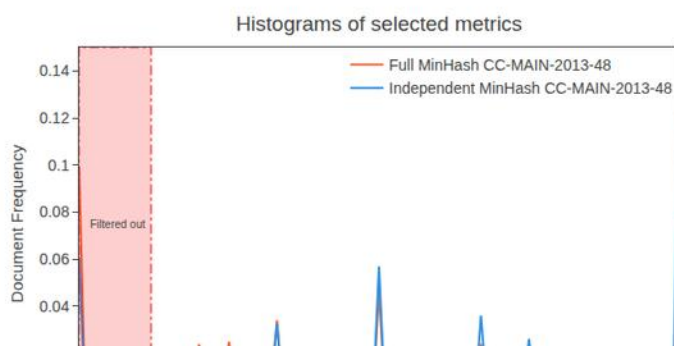
To develop new heuristic filters and select their thresholds we devised a systematic process:

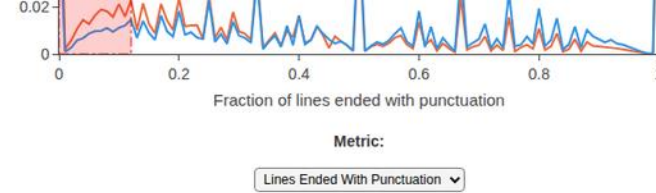
1. we started by collecting a very large list of high level statistics of our datasets (over **fifty** different metrics) ranging from common document-level metrics (e.g. number of lines, avg. line/word length, etc) to inter-document repetition metrics (inspired by MassiveText), on both a high quality and a lower quality web dataset;
2. we selected the metrics for which the Wasserstein distance between the two distributions (of the metric computed on each dataset) was larger;
3. we inspected the histograms of the two distributions and empirically chose a threshold that would make the lower quality dataset more closely resemble the higher quality one on this metric;
4. we validated the resulting filter (metric-threshold pair) by using it on a reference dataset and running small ablations.

Due to our (new) assumption that global MinHash greatly upsamples lower quality data in the oldest dumps, we computed metrics on both the independently MinHashed and the (worse quality) global MinHashed versions of the 2013-48 and 2015-22 crawls (two older crawls). We then compared the statistics at a macro level, by looking at the distribution of these metrics for each one.

Perhaps not too surprisingly given our findings for deduplication, we found significant disparities in most of the metrics for the two deduplication methods. For instance, the `line-char-duplicates` metric (nb. of characters in duplicated lines / nb. characters), roughly doubled from the independent dedup (0.0053 for 2015-22 and 0.0058 for 2013-48), to the global dedup (0.011 for 2015-22 and 0.01 for 2013-48), indicating that the latter had higher inter-document repetition.

Following the process listed above for these datasets yielded **seventeen** candidate metric-threshold pairs. In the image below, you can see three of these histograms:

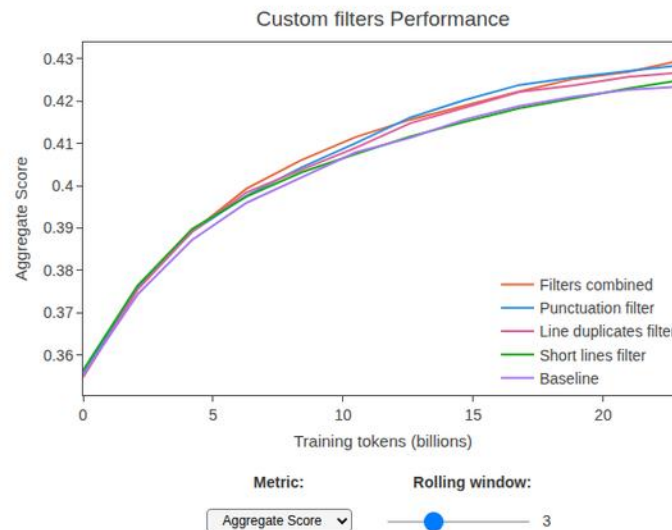




As an example, we inspected the histograms of "fraction of lines ending with punctuation" (see the image above) and observed an increased document density of global MinHash at around 0.12. We then filtered with this threshold and found that the removed data had a higher amount of short lists or consisted of only document layout text ("Home", "Sign up", etc).

We then assessed the effectiveness of these seventeen newly created filters, by conducting several of our 28 billion tokens ablation runs on the 2019-18 crawl. Out of all those runs, we identified **three** filters (the ones based on the histograms above) that demonstrated the most significant improvements on the aggregate score:

- Remove documents where the fraction of lines ending with punctuation ≤ 0.12 (10.14% of tokens removed) — vs the 30% from the original C4 terminal punct filter
- Remove documents where the fraction of characters in duplicated lines ≥ 0.1 (12.47% of tokens removed) — the original MassiveText threshold for this ratio is ≥ 0.2
- Remove documents where the fraction of lines shorter than 30 characters ≥ 0.67 (3.73% of tokens removed)
- When applying the three together, ~22% of tokens were removed.

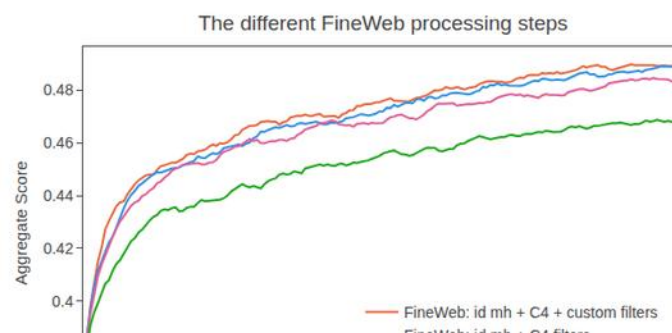


These filters allowed us to further improve performance and to, notably, surpass the C4 dataset performance while providing a much larger dataset at the same time.

The final 🍷 FineWeb dataset

The final 🍷 FineWeb dataset comprises 15T tokens and includes the following previously mentioned steps, in order, each providing a performance boost on our group of benchmark tasks:

- base filtering
- independent MinHash deduplication per dump
- a selection of C4 filters
- our custom filters (mentioned in the previous section)



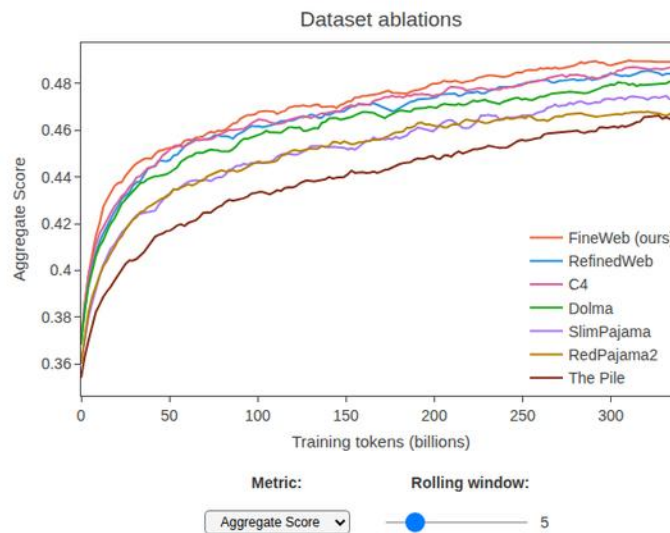


COMPARISONS WITH OTHER WEB-SCALE DATASETS

We compared 🍷 [FineWeb](#) with the following datasets that are usually considered the highest quality openly accessible web-scale datasets (we also indicate for each the approximate number of tokens in the public version of the dataset):

- [RefinedWeb](#) (500B tokens) [22]
- [C4](#) (172B tokens) [30]
- [Dolma v1.6](#) (3T tokens) (the CommonCrawl part) [32] ¹³
- [The Pile](#) (340B tokens) [33]
- [SlimPajama](#) (627B tokens) [34]
- [RedPajama2](#) (20T tokens) [35] (deduplicated)
- and our new 🍷 [FineWeb](#) (15T tokens) (this report)

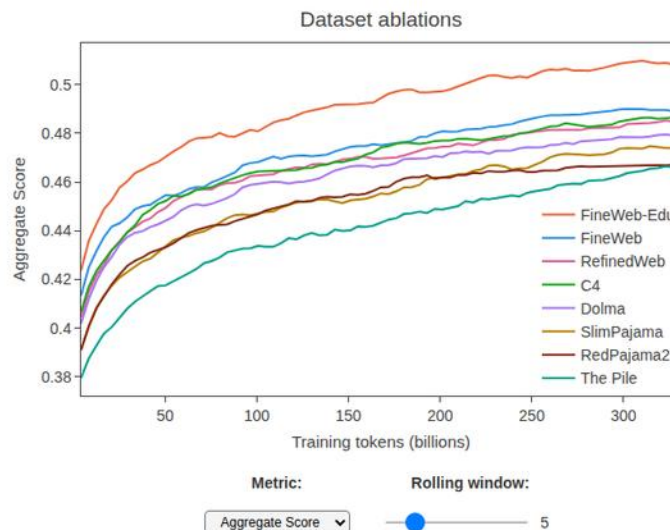
You will find the 350B-tokens-trained ablation models openly accessible and gathered in [this collection](#). We have uploaded checkpoints at every 1000 training steps. You will also find our full [evaluation results here](#).



🍷 [FineWeb](#) is thus – to the best of our knowledge – the open dataset leading to the current highest model performances while allowing to train on several trillion tokens.



FineWeb-Edu



 **FineWeb-Edu** is an additional development of FineWeb that we are excited to introduce in this tech report and openly release.  FineWeb-Edu is based on a new approach that has recently emerged for filtering LLM training datasets: using synthetic data to develop classifiers for identifying educational content. This technique was notably used in the trainings of Llama 3[1] and Phi3[36], but its large-scale impact on web data filtering has, in our opinion, thus far not been publicly explored to its full potential.

The popular Phi3 models were trained on 3.3 and 4.8 trillion tokens, with the paper[36] stating:

Our training data consists of heavily filtered publicly available web data (according to the 'educational level') from various open internet sources, as well as synthetic LLM-generated data.

Similarly, Llama 3 blog post[37] notes:

We found that previous generations of Llama are good at identifying high-quality data, so we used Llama 2 to help build the text-quality classifiers that are powering Llama 3.

However, these classifiers and filtered datasets are not publicly available. To further enhance

 FineWeb's quality, we developed an educational quality classifier using annotations generated by Llama-3-70B-Instruct to create  **FineWeb-Edu**.

Annotating for educational quality at scale

We used Llama-3-70B-Instruct to annotate 500k samples from  FineWeb, scoring each for their educational quality on a scale from 0 to 5.

We explored various prompt formats to automatically extract an educational score using an LLM and found that the additive scale by Yuan et al. [38] worked best. This scale allows the LLM to reason about each additional point awarded, unlike the single-rating Likert scale which fits samples into predefined boxes. Then, to avoid the LLM favoring highly technical pages like arXiv abstracts and submissions, we focused on grade-school and middle-school level knowledge. By setting a threshold of 3 (on a scale of 0 to 5) during the filtering process, we were able to also retain some high-level educational pages.

Below is an extract from a web page. Evaluate whether the page has a high educational value and could be useful in an educational setting for teaching from primary school to grade school levels using the additive 5-point scoring system described below. Points are accumulated based on the satisfaction of each criterion:

- Add 1 point if the extract provides some basic information relevant to educational topics, even if it includes some irrelevant or non-academic content like advertisements and promotional material.
- Add another point if the extract addresses certain elements pertinent to education but does not align closely with educational standards. It might mix educational content with non-educational material, offering a superficial overview of potentially useful topics, or presenting information in a disorganized manner and incoherent writing style.
- Award a third point if the extract is appropriate for educational use and introduces key concepts relevant to school curricula. It is coherent though it may not be comprehensive or could include some extraneous information. It may resemble an introductory section of a textbook or a basic tutorial that is suitable for learning but has notable limitations like treating concepts that are too complex for grade school students.
- Grant a fourth point if the extract is highly relevant and beneficial for educational purposes for a level not higher than grade school, exhibiting a clear and consistent writing style. It could be similar to a chapter from a textbook or a tutorial, offering substantial educational content, including exercises and solutions, with minimal irrelevant information, and the concepts aren't too advanced for grade school students. The content is coherent, focused, and valuable for structured learning.
- Bestow a fifth point if the extract is outstanding in its educational value, perfectly suited for teaching either at primary school or grade school. It follows detailed reasoning, the writing style is easy to follow and offers profound and thorough insights into the subject matter, devoid of any non-educational or complex content.

The extract: <extract>.

After examining the extract:

- Briefly justify your total score, up to 100 words.
- Conclude with the score using the format: "Educational score: <total points>".

Prompt used for Llama3 annotations of the educational score, also available [here](#).

In terms of open-weight models to use for annotating the data, we experimented with several models including Mixtral-8x7B-Instruct and Mixtral-8x22B-Instruct, Llama-3-70B-Instruct as well as a jury gathering the scores from these three models[39]. In our experiments we found that using Llama3 alone gave the most reliable results.

Training a classifier

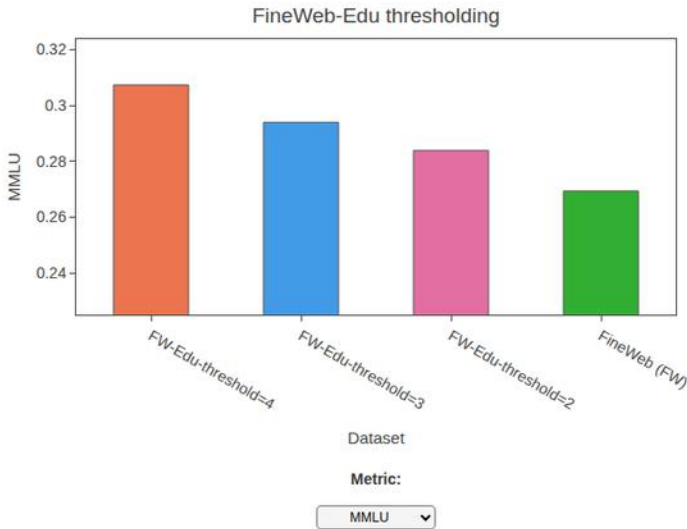
To scale our annotations to the trillions of tokens in FineWeb, we used the Llama3-70B annotations to train a small classifier. The model we used was a Snowflake-arctic-embed embedding model with a classification head with a single regression output on top of it. We trained this model on the 450,000 Llama 3 annotations for 20 epochs with a learning rate of 3e-4, freezing the embedding and encoder layers. We saved the checkpoint with the highest F1 score on our held-out validation set of 45k samples, treating Llama 3 annotations as ground-truth. After training, we rounded the scores to integers from 0 to 5.

We then converted the problem to a binary classification task by using a fixed threshold to determine if a file is educational. With a threshold of 3, the model achieved an F1 score of 82% on the validation set, indicating strong performance in distinguishing high-quality educational content.

The classifier is available at: [HuggingFaceFW/fineweb-edu-classifier](#). The training and

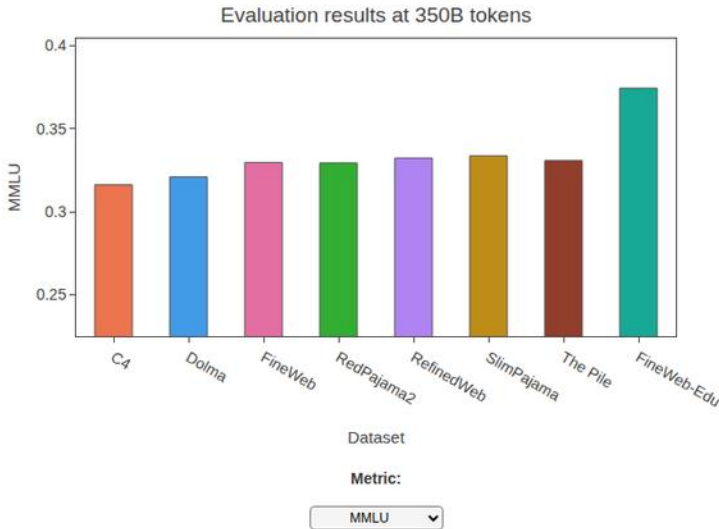
Filtering and results

We applied the classifier to the 15T tokens of 🍷 FineWeb, a process that required 6,000 H100 GPU hours. We investigated the impact of using different thresholds for the filtering and found that using a threshold of 3 gave the best overall results. Although using a threshold higher than 3 improves performance on knowledge and reasoning intensive benchmarks, it significantly degrades performance on HellaSwag and PIQA. The plot below shows the performance of each threshold compared to FineWeb on six different benchmarks; it uses a 1.82B model trained on 8B tokens.



Note: this ablation was conducted on 8B tokens from the 2024-10 dump for both the FineWeb and FineWeb-Edu subsets, which might not be representative of the entire dataset. The next ablation shows that the findings for threshold 3 hold on a longer run of 350B tokens from all FineWeb dumps, except for HellaSwag, where we noticed a slight performance degradation.

We built 🍷 FineWeb-Edu by filtering out samples with scores lower than 3. This removed 92% of the dataset, leaving us with 1.3 trillion educational tokens. To evaluate the effectiveness of this filtering at a larger scale, we conducted an ablation using a 1.82B model trained on 350 billion tokens, similar to the FineWeb filtering ablation mentioned above:



Here are the key highlights of the ablation results above:

- 🍷 FineWeb-Edu surpasses 🍷 FineWeb and all other open web datasets, with remarkable improvements on educational benchmarks such as MMLU, ARC, and OpenBookQA.
- It achieves the same performance with significantly less data, requiring 10x fewer tokens compared to C4 and Dolma to match MMLU results.
- This demonstrates the effectiveness of using classifiers trained on LLM annotations for large-scale data filtering.

Given that a threshold of 2 also demonstrated strong performance while retaining more data, we are releasing an additional dataset filtered with this threshold, containing 5.4 trillion tokens

are releasing an additional dataset filtered with this threshold containing 67 million tokens under [HuggingFaceFW/fineweb-edu-score-2](#).

You can find the two datasets along with the classifier used for the filtering in this [collection](#).

Bonus: CommonCrawl over time

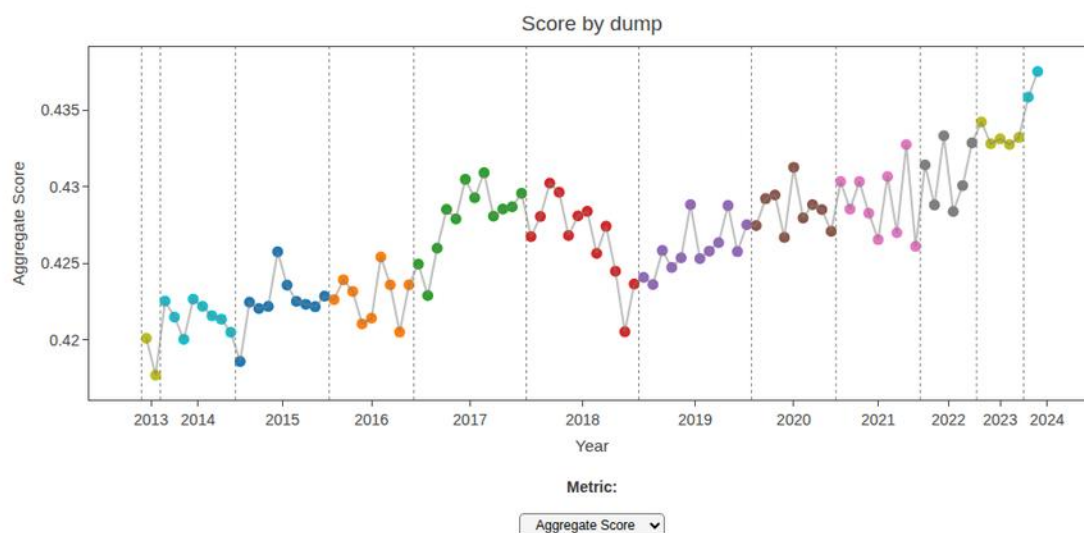
Just like fine wine, not all crawls are created equal.

While ablating filtering steps, we noticed that certain crawls outperformed others by a significant margin. We decided to investigate this phenomenon.

Benchmark performance by crawl

For each crawl, we trained two 1.8B models on 27 billion tokens randomly sampled from that crawl's data (after the base filtering and MinHash deduplication steps), where each run had a different random 27BT sampling of this data. We trained 192 such models, totaling over 60 thousand H100 GPU-hours. We subsequently took the last 3 checkpoints for both runs and plotted the average of these 6 data points per crawl.

The plot below clearly shows that some dumps perform far worse than others. Each year has a different color, and the number of crawls per year also varies.



We investigated possible causes for this behaviour such as changes in the most common URLs of each dump, as well as potential benchmark contamination, but could not find any conclusive explanation. We leave further investigation for future work.

Synthetic data

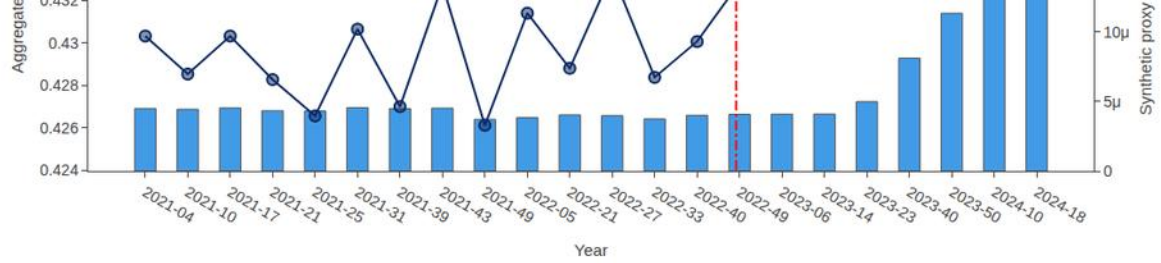
We wondered if the strong performance of the last few crawls could be, in part, attributed to the presence of a larger quantity of synthetic data (data generated by LLMs). Such a change would not be surprising due to the recent increase in popularity of LLMs, notably of ChatGPT.

Since, to the best of our knowledge, there is no foolproof method to detect synthetic data, we opted to use a proxy metric: we measured the frequency of the following words in each crawl: "delve", "as a large language model", "it's important to note", "rich tapestry", "intertwined", "certainly!", "dive into", all of which are commonly used by ChatGPT.

It is important to note that not all samples containing one of these phrases were necessarily generated by ChatGPT (and also that many ChatGPT generated samples do not contain any of these phrases), but assuming that the amount of synthetic data were to not change across crawls, one would expect these frequencies to remain approximately constant over time.

The results are shown in the following plot:





While the frequency remained approximately constant until 2023-14 (ChatGPT was released at the end of 2022), we find a steep increase of our proxy metric in recent crawls. While this simple test is not enough to conclude that ChatGPT completions and other synthetic data is improving the quality of the most recent crawl, it at the very least does not seem to drastically harm it.

We expect to continue seeing increasing quantities of synthetic data on new CC crawls. However, while for relatively small trainings this data does not seem to harm performance (and might actually improve it), it is not clear that this holds for much larger trainings.

Conclusion and looking forward

Through our open science efforts we hope to keep shining a light on the black box that is the training of high performance large language models as well as to give every model trainer the ability to create state-of-the-art LLMs. We are excited to continue iterating on FineWeb and to release increasingly better filtered subsets of web data, in a fully open and reproducible manner.

In the short term, we are looking forward to applying the learnings from (English) FineWeb to other languages. While English currently dominates the LLM landscape, we believe that making high quality web data in other languages as accessible as possible would be incredibly impactful.

In a nutshell: the future is bright and exciting for studying the science of creating datasets at scale and in the open 😊.

Citation

For attribution in academic contexts, please cite this work as

Penedo, et al., "The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale", 2024.

BibTeX citation

```
@Inproceedings{
  penedo2024the,
  title={The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale},
  author={Guilherme Penedo and Hynek Kydl{\v{e}}k and Loubna Ben allal and Anton Lozhkov and Margaret Mitchell and Colin Raffel and Leandro Von Werra and Thomas Wolf},
  booktitle={The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track},
  year={2024},
  url={https://openreview.net/forum?id=n65Ckn2QaG}
}
```

Footnotes

- Note that the size changes from crawl to crawl. Note also that we use "dump" or "crawl" interchangeability in this report. [↩]
- We have not processed these 3 older crawls. [↩]
- Note that this report is focused on the special field of web-scale datasets ("web-scale" typically meaning >100 billion tokens obtained from the web) used to pretrain a Large Language Model (by pretraining we mean the very first step in the training of a model, starting from random weights). We don't pretend to cover any other field of dataset creation nor that the lessons or hypothesis we develop in this document can extend to any field besides this specific field. [↩]
- Even though as we mentioned above the notion of "clean" is so ill-defined that it should probably not be seen as equivalent to wikipedia-type of text [↩]
- "Small" in comparison to standard sizes of today's LLMs, i.e. small in comparison to 7-70 billion parameters. In this work "small" means about 1-2 billion parameters [↩]
- In particular we suspect that it keeps too much boilerplate content and navigation menus. [↩]
- We used traillatura default options with `favour_precision=True`. [↩]
- As everywhere in this report: this is the number of tokens when tokenized with the gpt2 tokenizer [↩]
- Note that here, even when we discuss "fuzzy" deduplication, we are only employing methods that operate on character/word matches, aka surface-level text. A more complex concept of deduplication is concerned with "semantic" deduplication: comparing/removing texts which are relative to the same concepts and use for instance synonyms or paraphrasing. We don't discuss these topics here but note that they can be important in the field of large-scale synthetic data generation for instance (see our [Cosmopedia release](#) on this topic) [↩]
- Our units are "words", computed in the [MinHash processing function](#) with a [language-specific word tokenizer](#). [↩]
- While there may be documents in *originally kept data* similar to documents in *originally removed data*, we estimate the overlap to be small (around 4 billion tokens). [↩]

12. Note that these ablation models are trained only on data from this dump so it's considered independently of all the other dumps. [\[↗\]](#)
13. There is a newer version of Dolma, v1.7, which is smaller [\[↗\]](#)

References

1. Llama 3 Model Card [\[link\]](#)
AI@Meta., 2024.
2. Mixtral of Experts
Jiang, A.Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D.S., Casas, D.d.I., Hanna, E.B., Bressand, F., Lengyel, G., Bour, G., Lample, G., Lavaud, L.R., Saulnier, L., Lachaux, M., Stock, P., Subramanian, S., Yang, S., Antoniak, S., Scao, T.L., Gervet, T., Lavril, T., Wang, T., Lacroix, T. and Sayed, W.E., 2024.
3. Language Models are Unsupervised Multitask Learners
Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I., 2019.
4. DataTrove: large scale data processing [\[link\]](#)
Penedo, G., Kydlíček, H., Cappelli, A., Sasko, M. and Wolf, T., 2024. GitHub repository. GitHub.
5. A Survey on Data Selection for Language Models
Albalak, A., Elazar, Y., Xie, S.M., Longpre, S., Lambert, N., Wang, X., Muennighoff, N., Hou, B., Pan, L., Jeong, H., Raffel, C., Chang, S., Hashimoto, T. and Wang, W.Y., 2024.
6. Measuring Data
Mitchell, M., Luccioni, A.S., Lambert, N., Gerchick, M., McMillan-Major, A., Ozoani, E., Rajani, N., Thrush, T., Jernite, Y. and Kiela, D., 2023.
7. A Pretrainer's Guide to Training Data: Measuring the Effects of Data Age, Domain Coverage, Quality, & Toxicity
Longpre, S., Yauney, G., Reif, E., Lee, K., Roberts, A., Zoph, B., Zhou, D., Wei, J., Robinson, K., Mimno, D. and Ippolito, D., 2023.
8. CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data
Wenzek, G., Lachaux, M., Conneau, A., Chaudhary, V., Guzmán, F., Joulin, A. and Grave, E., 2019.
9. Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research
Soldaini, L., Kinney, R., Bhagia, A., Schwenk, D., Atkinson, D., Authur, R., Bogin, B., Chandu, K., Dumas, J., Elazar, Y., Hofmann, V., Jha, A.H., Kumar, S., Lucy, L., Lyu, X., Lambert, N., Magnusson, I., Morrison, J., Muennighoff, N., Naik, A., Nam, C., Peters, M.E., Ravichander, A., Richardson, K., Shen, Z., Strubell, E., Subramani, N., Tafjord, O., Walsh, P., Zettlemoyer, L., Smith, N.A., Hajishirzi, H., Beltagy, I., Groeneveld, D., Dodge, J. and Lo, K., 2024.
10. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference
Chiang, W., Zheng, L., Sheng, Y., Angelopoulos, A.N., Li, T., Li, D., Zhang, H., Zhu, B., Jordan, M., Gonzalez, J.E. and Stolca, I., 2024.
11. Training language models to follow instructions with human feedback
Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J. and Lowe, R., 2022.
12. Training Compute-Optimal Large Language Models
Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D.d.L., Hendricks, L.A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G.v.d., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J.W., Vinyals, O. and Sifre, L., 2022.
13. CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge [\[link\]](#)
Talmor, A., Herzig, J., Lourie, N. and Berant, J., 2019. Proceedings of the 2019 Conference of the North (A)merican Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 4149–4158. Association for Computational Linguistics. DOI: 10.18653/v1/N19-1421
14. HellaSwag: Can a Machine Really Finish Your Sentence? [\[link\]](#)
Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A. and Choi, Y., 2019. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 4791–4800. Association for Computational Linguistics. DOI: 10.18653/v1/P19-1472
15. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering
Mihaylov, T., Clark, P., Khot, T. and Sabharwal, A., 2018. EMNLP.
16. PIQA: Reasoning about Physical Commonsense in Natural Language
Bisk, Y., Zellers, R., Bras, R.L., Gao, J. and Choi, Y., 2019.
17. SocialQA: Commonsense Reasoning about Social Interactions
Sap, M., Rashkin, H., Chen, D., LeBras, R. and Choi, Y., 2019.
18. WinoGrande: An Adversarial Winograd Schema Challenge at Scale
Sakaguchi, K., Bras, R.L., Bhagavatula, C. and Choi, Y., 2019.
19. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge
Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C. and Tafjord, O., 2018.
20. Measuring Massive Multitask Language Understanding
Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D. and Steinhardt, J., 2021.
21. Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction [\[link\]](#)
Barbaresi, A., 2021. Proceedings of the Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations, pp. 122–131. Association for Computational Linguistics.
22. The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only
Penedo, G., Malartic, Q., Hesslow, D., Cojocar, R., Cappelli, A., Alobeidli, H., Pannier, B., Almazroue, E. and Launay, J., 2023.
23. Bag of Tricks for Efficient Text Classification
Joulin, A., Grave, E., Bojanowski, P. and Mikolov, T., 2016. arXiv preprint arXiv:1607.01759.
24. FastText.zip: Compressing text classification models
Joulin, A., Grave, E., Bojanowski, P., Douze, M., Jegou, H. and Mikolov, T., 2016. arXiv preprint arXiv:1612.03651.
25. Scaling Language Models: Methods, Analysis & Insights from Training Gopher
Rae, J.W., Borgeaud, S., Cai, T., Millican, K., Hoffmann, J., Song, F., Aslanides, J., Henderson, S., Ring, R., Young, S., Rutherford, E., Hennigan, T., Menick, J., Cassirer, A., Powell, R., Driessche, G.v.d., Hendricks, L.A., Rauh, M., Huang, P., Glaese, A., Welbl, J., Dhathathri, S., Huang, S., Uesato, J., Mellor, J., Higgins, I., Creswell, A., McAleese, N., Wu, A., Elsen, E., Jayakumar, S., Buchatskaya, E., Budden, D., Sutherland, E., Simonyan, K., Paganini, M., Sifre, L., Martens, L., Li, X.L., Kuncoro, A., Nematzadeh, A., Gribovskaya, E., Donato, D., Lazaridou, A., Mensch, A., Lespiau, J., Tsipoukelli, M.,

- Grigorev, N., Fritz, D., Sottiaux, T., Pajarskas, M., Pohlen, T., Gong, Z., Toyama, D., d'Autume, C.d.M., Li, Y., Terzi, T., Mikulik, V., Babuschkin, I., Clark, A., Casas, D.d.L., Guy, A., Jones, C., Bradbury, J., Johnson, M., Hechtman, B., Weidinger, L., Gabriel, I., Isaac, W., Lockhart, E., Osindero, S., Rimell, L., Dyer, C., Vinyals, O., Ayoub, K., Stanway, J., Bennett, L., Hassabis, D., Kavukcuoglu, K. and Irving, G., 2022.
26. Deduplicating Training Data Makes Language Models Better
Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C. and Carlini, N., 2022.
27. Quantifying Memorization Across Neural Language Models
Carlini, N., Ippolito, D., Jagielski, M., Lee, K., Tramer, F. and Zhang, C., 2023.
28. Scaling Data-Constrained Language Models
Muennighoff, N., Rush, A.M., Barak, B., Scao, T.L., Piktus, A., Tazi, N., Pyysalo, S., Wolf, T. and Raffel, C., 2023.
29. Scaling Laws and Interpretability of Learning from Repeated Data
Hernandez, D., Brown, T., Conerly, T., DasSarma, N., Drain, D., El-Showk, S., Elhage, N., Hatfield-Dodds, Z., Henighan, T., Hume, T., Johnston, S., Mann, B., Olah, C., Olsson, C., Amodel, D., Joseph, N., Kaplan, J. and McCandlish, S., 2022.
30. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer
Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W. and Liu, P.J., 2023.
31. LLaMA: Open and Efficient Foundation Language Models
Touvron, H., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E. and Lample, G., 2023.
32. Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research
Soldaini, L., Kinney, R., Bhagia, A., Schwenk, D., Atkinson, D., Authur, R., Bogin, B., Chandu, K., Dumas, J., Elazar, Y., Hofmann, V., Jha, A.H., Kumar, S., Lucy, L., Lyu, X., Lambert, N., Magnusson, I., Morrison, J., Muennighoff, N., Naik, A., Nam, C., Peters, M.E., Ravichander, A., Richardson, K., Shen, Z., Strubell, E., Subramani, N., Tafjord, O., Walsh, P., Zettlemoyer, L., Smith, N.A., Hajishirzi, H., Beltagy, I., Groeneveld, D., Dodge, J. and Lo, K., 2024. arXiv preprint.
33. The (P)ile: An 800{GB} dataset of diverse text for language modeling
Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N. and others., 2020. arXiv preprint arXiv:2101.00027.
34. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama [\[link\]](#)
Soboleva, D., Al-Khateeb, F., Myers, R., Steeves, J.R., Hestness, J. and Dey, N., 2023.
35. RedPajama: an Open Dataset for Training Large Language Models [\[link\]](#)
Computer, T., 2023.
36. Phi-3 technical report: A highly capable language model locally on your phone
Abdin, M., Jacobs, S.A., Awan, A.A., Aneja, J., Awadallah, A., Awadalla, H., Bach, N., Bahree, A., Bakhtiari, A., Behl, H. and others., 2024. arXiv preprint arXiv:2404.14219.
37. Our responsible approach to Meta AI and Meta Llama 3 [\[link\]](#)
Meta., 2024.
38. Self-rewarding language models
Yuan, W., Pang, R.Y., Cho, K., Sukhbaatar, S., Xu, J. and Weston, J., 2024. arXiv preprint arXiv:2401.10020.
39. Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models
Verga, P., Hofstätter, S., Althammer, S., Su, Y., Piktus, A., Arkhangorodsky, A., Xu, M., White, N. and Lewis, P., 2024. arXiv preprint arXiv:2404.18796.